CS-499 Milestone Four: Enhancement Narrative

Enhancement Three: Databases

Michael King

7/31/2026

Artifact Description

The artifact for this milestone is the weight tracking Android mobile application I developed for **CS-360: Mobile Architecture and Programming.** I created this program in Java with the Android Studio platform. The app was originally designed to enable a user to create an account and log in, log and review weight info over time, set a goal weight, and receive a text message upon achieving that goal. All data, including user information, weight data, and the goal, were stored locally on the device in an SQLite database, accessed through an object that performed synchronous database queries on the UI thread.

This enhancement involved migrating the application's storage layer from local SQLite to a cloud-hosted MongoDB Atlas deployed on Amazon cloud infrastructure (exposed to the Android client as a RESTful API) driven by a Flask/PyMongo backend (containing the *app.py* script plus its dependencies and configuration files). The Android client was also rewritten to interact with the Python/Flask API over HTTP via Retrofit rather than directly access a local database. The improved artifact has two co-working parts: a Flask/PyMongo backend (*app.py* script plus its dependencies and configuration files) and an Android client (split into individual activities, a Retrofit client service interface, request, and response data classes, and an Android-managed token for session persistence).

Why This Artifact Was Selected

I chose to demonstrate this artifact in the Databases category because the original storage scheme used a single device, unauthenticated local data store that, frankly, could not scale beyond a single phone or more than a casual single user. It was exactly what a database enhancement was supposed to fix. Looking at the original implementation also led me to produce an even more specific justification for the database build. There was no user ID column in the SQLite weight entry table, so that each user would have been reading and writing to the same pool of records in the early system, not their own.

Components That Showcase Database Design Skills

Several specific aspects of the enhanced artifact demonstrate knowledge of database design and software development skills. The Mongoose schema is normalized into two collections, users and weight_entries, each indexed on the field used in querying: a unique index on username enforces account uniqueness at the database level, and an index on the username field in each document in the entries collection keeps user-by-user lookup fast as the dataset grows.

The RESTful API surface (registration, login, and complete CRUD on weight entries and goal weight) scopes each authenticated query to the account indicated by the identity stored in a JSON Web Token. Because of this, it never allows one account to read, update, or delete another account's information, immediately addressing the flaw identified in the original artifact.

By immediately identifying and rejecting ill-formed input before passing it to a database query, the server-side input validation in each route (username syntax, password length, date syntax, and a limited range of weight data) also guards against possible NoSQL injection and

style payloads like a token specifier instead of a string. Finally, the handling of credentials was rethought around server-side salted password hashes and encrypted, rather than plain-text, token storage on the Android client.

How This Artifact Was Improved

The artifact was improved to replace all of the persistence layers while keeping the functionality of the application the same from a user perspective; the boosted client still supports login, weight entry, goal setting, and SMS notifications the same as before, but all of those actions take place over an authenticated network API call, and actually require a verified indexed per-user-scoped database rather than an unauthenticated file stored on the device.

Course Outcome Alignment

The enhancement plan submitted in Module One listed this artifact's main course outcome as demonstrating mastery of and innovation in using techniques, skills, and tools in computing (namely NoSQL design, REST API creation, and authentication that warded against malicious methods). It provided a weaker combination of the secondary course outcome focused on understanding adversarial attacks and fixing design flaws. Both outcomes were achieved.

The completed migration involving the functioning Flask/PyMongo REST API with indexed collections, and an Android client built with Retrofit that calls that API asynchronously, and an authentication flow using JWT that links the two has fulfilled the primary course outcome. The secondary course outcome was also fulfilled, and in one respect strayed beyond the initially planned scope. The Module One plan aimed to demonstrate security practices through hashing passwords, parameterized queries, and token-based authentication in the abstract. The implementation proved to be an extra, unplanned security insight: the core artifact's

prior data entry source file did not scope weighing any permanently entered items by user at all, rather than as a data-isolation bug. Addressing this flaw entailed scoping every read, update, and delete operation to the authenticated user identity, rather than to a record's raw ID; this is, in spirit, a more concrete proactive step toward thwarting an attacker than the initial plan explicitly accounted for. All other revisions to the outcome and coverage plan are extraneous; Modules One and Two's targeted categories are still the appropriate match for this artifact.

Reflection

Enhancing the artifact made me think of the application as a distributed system, rather than a single machine with a local database file. Moving persistence off the device forced me to redesign the assumptions that had previously existed: each synchronous DAO call was now a network request, the record readable off disk had to be verifiable and authorized at each access, and the cleartext credential stored locally had to be hashed on the server and encrypted into a token on the client. Working through that evolution made clear the kinds of implicit assumptions that a single-machine, single-user design can avoid, and how those assumptions need to be explicitly codified and enforced once there could be more than one path to the data.

Challenges

Some challenges were encountered during implementation and testing, yet were solved by straightforward debugging rather than integrated into the design. There was a Flask route that used the same handler function name as another route already defined. Flask treats the handler as a unique ID for the route and prevented server startup with an observed assertion error that provided no information about the cause, and the route code was absent.

A MongoDB Atlas connection string copied directly from the Atlas console did not include the target database name and resulted in an error during startup. In addition to hardening the connection string against this by correcting it, the backend was modified to explicitly choose which database name to connect to in code rather than expecting to find the database name somewhere in an externally supplied connection string.

The theme used by the Android project, which was supplied by the IDE because the default Compose project template was selected, was not derived from an AppCompat-compatible theme, causing both activities to crash on startup with an IllegalStateException. Working out what was wrong with it involved tracing the crash back to the Android theme inheritance hierarchy, rather than to what otherwise seemed to be the failed activity code.

Manual testing uncovered a functional bug that was not apparent from the notification code. Since the “goal reached” check was reimplemented on every add/edit/delete and had no state to retain previous notifications, if a user stayed below their goal weight, they would receive a second congratulatory text on every subsequent add/edit/delete. The fix added “persistent state” to track the last notified goal weight, and only send a notification if the goal state actually transitioned from not met to met. A manual test sequence of reach goal, lower goal, higher goal, and reach goal again demonstrated the desired behavior: one notification per goal transition plus no false notifications.

Conclusion

This experience made it clear that the most important bugs are seldom apparent just from reading one function. The Flask endpoint collision, the theme inheritance crash, and the notification state bug didn’t come out until I did an actual end-to-end run of the code. That

reinforced the practical, valuable lesson of testing each layer separately: the API individually uses requests to run the whole client-to-server system as a live application, rather than just assuming that sane code will work well in its debut in a live environment.